

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

Laravel

El modelo de datos

Pepe Devesa



GENERALITAT VALENCIANA
Conselleria d'Educació, Investigació, Cultura i Esport



GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, CULTURA Y DEPORTE

Continguts

1 Parámetros de conexión a la base de datos **3**

2 Crear el modelo **4**

2.1 Seguir una nomenclatura uniforme 5

3 Operaciones sobre el modelo. Primeros pasos con Eloquent. **5**

3.1 Realizar búsquedas 5

3.2 Consultas con Eloquent 6

3.3 Usando la BD con QueryBuilder 6

3.4 Fichas de objetos individuales. 9

3.5 Inserciones 10

3.6 Modificaciones 11

3.7 Borrados 12

Una vez vistas dos de las tres patas en que se sustenta el patrón **MVC** (las vistas y los controladores), en esta sección abordaremos la tercera de ellas: **el modelo de datos**. Con esto, trataremos algunas cuestiones importantes sobre cómo gestiona Laravel el acceso a bases de datos, y qué mecanismos ofrece para sincronizar los datos de nuestra aplicación con los documentos o registros de una base de datos, así como para generar automáticamente la estructura de tablas y campos de la base de datos a partir del modelo de la aplicación.

1 Parámetros de conexión a la base de datos

Una de las primeras cosas que debemos hacer para configurar el acceso a la base de datos en nuestro proyecto es establecer los parámetros con los que conectar a dicha base de datos: *nombre del servidor, usuario, contraseña, etc*. Estos parámetros se definen en el archivo **.env** para cada entorno de despliegue de la aplicación (recuerda que este archivo no se sube a Git, por lo que cada entorno tendrá el suyo). Dentro de este archivo, debemos modificar las siguientes variables de entorno:

- `DB_CONNECTION`: tipo de SGBD a usar
- `DB_HOST`: dirección o IP del SGBD (127.0.0.1 para conexión local)
- `DB_PORT`: puerto por el que el SGBD estará escuchando. Por ejemplo, el puerto por defecto para MySQL es 3306
- `DB_DATABASE`: nombre de la base de datos a la que conectar
- `DB_USERNAME`: login del usuario para conectar
- `DB_PASSWORD`: password del usuario para conectar

En cuanto al primer parámetro (DB_CONNECTION), aquí tenemos un listado de los sistemas más habituales, junto con sus puertos por defecto que podemos utilizar en DB_PORT:

Id SGBD	Nombre SGBD	Puerto por defecto
mysql	MySQL/MariaDB	3306
oracle	Oracle	1521
pgsql	PostgreSQL	5432
sqlsrv	SQL Server	1433
sqlite	SQLite	-

Ahora, y suponiendo, que ya tenemos la estructura de tablas creada en la base de datos, vamos a ver qué mecanismos ofrece Laravel para acceder a estos datos de forma sencilla desde la aplicación. Veremos

cómo definir el modelo de datos asociado a cada tabla, y cómo manipular estos datos empleando el `ORM Eloquent`, incorporado con Laravel.

2 Crear el modelo

La idea es crear una clase por cada tabla que tengamos en nuestra base de datos, para así interactuar con la tabla a través de dicha clase asociada. Para crear esta clase modelo, utilizamos la opción `make:model` del comando `php artisan`. Le pasaremos como parámetro adicional el nombre de la clase a crear. Por ejemplo, para el caso de nuestra biblioteca, podemos crear así el modelo Libro:

```
php artisan make:model Libro
```

Por convención, los modelos se crean con un nombre en singular, empezando por mayúscula, y se ubican en la carpeta `app\Models`. La estructura básica del modelo es algo así:

```
    <?php

    namespace App\Models;

    use Illuminate\Database\Eloquent\Factories\HasFactory;
    use Illuminate\Database\Eloquent\Model;

    class Libro extends Model
    {
        use HasFactory;
    }
    ?>
```

En nuestro caso, vamos también a utilizar el modelo de `usuario` que ya existe en la carpeta `app\Models`, aunque lo renombraremos de `User` a `Usuario`:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Usuario extends Model
{
    ...
```

Automáticamente, se asocia este modelo a una tabla con el mismo nombre, pero en plural y en minúscula, por lo que los modelos anteriores estarían asociados a unas tablas **libros y usuarios en la base de datos**, respectivamente. En caso de que no queramos que sea así, definimos una propiedad `$table` en la clase con el nombre que queramos que tenga la tabla asociada. Por ejemplo:

```php
class Libro extends Model
{
    protected $table = 'mislibros';
}
```

2.1 Seguir una nomenclatura uniforme

Recuerda que, de sesiones anteriores, hemos comentado la recomendación/necesidad de seguir una nomenclatura uniforme en los modelos, controladores y vistas. Así, para el **modelo Libro** ya tendríamos:

- `controlador` asociado `LibroController`.
- Las `vistas` se definirían en la subcarpeta `resources/views/libros`, con los nombres correspondientes a cada método del controlador (por ejemplo, `index.blade.php`, o `show.blade.php`).

3 Operaciones sobre el modelo. Primeros pasos con Eloquent.

Eloquent es el ORM incorporado por defecto en Laravel. Un `ORM (Object Relational Mapping)` **es una herramienta que permite establecer una relación entre los registros de una tabla de la base de datos y los objetos de una clase (de PHP en nuestro caso), de forma que los datos de la base de datos se convierten a objetos PHP y viceversa**. Además, Eloquent implementa el patrón `Active Record`, que añade a las clases métodos como `save`, `update`, `delete...` que permiten interactuar con la base de datos para insertar, modificar o borrar registros asociados a objetos, respectivamente.

3.1 Realizar búsquedas

Una vez creado el modelo, y aunque esté vacío, ya podemos utilizarlo en los controladores para acceder a los datos. Basta con importar la clase correspondiente `(con use)`, y utilizar los métodos que se heredan de `Model`. Por ejemplo, el método `get` o `all` permite obtener los registros de la tabla, convertidos a objetos. Así es como obtendríamos todos los libros de la tabla desde un controlador:

```php
use App\Models\Libro;


class LibroController extends Controller
{
    public function index()
```

```
    {
        $libros = Libro::get();
        return view('libros.index', compact('libros'));
    }
}
```

3.2 Consultas con Eloquent

```
$libros = Libro::all();        // Devuelve todos los artículos
$libros = Libro::find($id);       // Devuelve el artículo con ese $id
$libros = Libro::findOrFail($id); // Error 404 si el artículo no existe
$libros = Libro::where('id', '>', 100)->get();  // Select con where
$libros = Libro::where('id', '>', 100)->take(10)->get();  // Select con
   where y take
$libros = Libro::max('id');           // Devuelve el último id asignado
```

También podriamos utilizar el sql convencional. Hay que añadir `use Illuminate\Support\Facades\DB;`

```
use Illuminate\Http\Request;
use App\Models\Libro;

use Illuminate\Support\Facades\DB;


class LibroController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        //
        $libros = DB::select('select * from libros');
        //$libros = Libro::all();
        return view('libros.index', compact('libros'));

    }
```

3.3 Usando la BD con QueryBuilder

El grado de abstracción de QueryBuilder es mucho menor que el de Eloquent. Es decir, estaremos CASI escribiendo SQL, sin llegar a hacerlo.

Algunos ejemplos de uso te darán una pista de qué tipo cosas se pueden hacer con QueryBuilder:

```
$users = DB::table("users")->get();
$users = DB::table("users")->where("name", "=", "Ana")->first();
$users = DB::table("users")->where("edad", ">=", 18)->orderBy("apellidos")
    ;
$maxId = DB::table("users")->max("id");
$existe = DB::table("users")->where("id", "=", $id)->exists();
$users = DB::table("users")->select("nombre, apellidos as apell")->get();
```

Lo que obtenemos es un array de objetos, por lo que deberemos acceder a sus propiedades como tales.

Por ejemplo, si queremos mostrar los títulos de los libros en una vista Blade, haríamos algo como esto:

```
@forelse($libros as $libro)
    {{ $libro->titulo }}
@endforelse
```

Alternativamente, también podemos obtener una consulta filtrada, especificando con el método `where` la condición que deben cumplir los registros a obtener.

Por ejemplo, así obtendríamos los libros cuyo precio sea inferior a `10` euros:

```
$libros = Libro::where('precio', '<', 10)->get();
```

De este otro modo obtendríamos libros con precio inferior a 10 euros y superior a 5 euros, de modo que podemos combinar condiciones:

```
$libros = Libro::where('precio', '<', 10)
    ->where('precio', '>', 5)->get();
```

Sobre estas consultas base podemos aplicar una serie de añadidos. Por ejemplo, podemos querer ordenar los libros por título, para lo que haríamos esto en el controlador:

```
$libros = Libro::orderBy('titulo')->get();
```

El método orderBy admite un segundo parámetro que indica el sentido de la ordenación. Por defecto es ASC (ascendente), pero también puede ser DESC:

```
$libros = Libro::orderBy('titulo', 'DESC')->get();
```



Ventajas de QueryBuilder sobre SQL
Como ves, QueryBuilder te permite construir sentencias SQL sin necesidad de escribir SQL.
La ventaja de esto es triple:

1. *No tendremos que depurar nuestros errores sintácticos en SQL*, con el ahorro de tiempo que eso conlleva.
2. El SQL generado será 100% compatible con el gestor de base de datos que estemos utilizando. Si escribimos SQL en crudo, tendremos que adaptarlo al dialecto de nuestro gestor de base de datos. Y, si cambiamos de gestor, habrá que revisar todas las sentencias SQL para adaptarlas de nuevo. Todo esto lo evita QueryBuilder, puesto que hace esa *adaptación por nosotros*.
3. Es imposible que suframos un ataque por *inyección de código*, puesto que QueryBuilder no lo permitirá.

3.3.1 Paginaciones de resultados

Si queremos paginar los resultados obtenidos debemos, por un lado, cuando obtengamos el listado desde el controlador, indicar con paginate cuántos registros queremos por página:

```
public function index()
{
    $libros = Libro::paginate(5);
    return view('libros.index', compact('libros'));
}
```

Después, en la vista asociada (libros.index en el ejemplo anterior), podemos emplear el método `links` para que muestre los botones de paginación en el lugar deseado:

```
@forelse($libros as $libro)
    {{ $libro->titulo }}
@endforelse

{{ $libros->links() }}
```

Si queremos ordenar el listado, podemos emplear orderBy u orderByDesc, pasándole como parámetro el nombre del campo por el que ordenar, antes de la paginación. Podemos, incluso, ordenar por múltiples criterios concatenados:

```
public function index()
{
    $libros = Libro::orderByAsc('titulo')
```

```
        ->orderByAsc('editorial')
        ->paginate(5);
    return view('libros.index', compact('libros'));
}
```

3.3.2 Paginaciones desde Laravel 8

En la versión 8 de Laravel se ha cambiado el estilo de los botones de paginación, empleando el del framework Tailwind CSS. Si queremos seguir utilizando los de Bootstrap, debemos añadir una de estas líneas en el método `boot` del provider `App\Providers\AppServiceProvider`, dependiendo de la versión de Bootstrap que queramos usar (por defecto se usará la 5 en la versión actual de los apuntes):

```
// Bootstrap 4
Paginator::useBootstrap();
// Bootstrap 5
Paginator::useBootstrapFive();
```

Además, debemos incorporar la cláusula `use` para localizar el elemento `Paginator`:

```
use Illuminate\Pagination\Paginator;
```

3.4 Fichas de objetos individuales.

Una operación bastante habitual es mostrar una ficha de un objeto a partir de un listado, haciendo clic en el título o alguna parte visible de ese objeto. Por ejemplo, si queremos ver los datos de un libro a partir de un listado con sus títulos, podemos hacer algo como esto en la plantilla Blade:

```
@forelse($libros as $libro)
    <li><a href="{{ route('libros.show', $libro) }}">
            {{ $libro->titulo }}
    </a></li>
@endforelse
```

Vemos que hemos utilizado el método `route` para indicar la ruta a seguir, con un segundo parámetro, que en este caso es el objeto concreto de esa fila. **Laravel automáticamente lo reemplazará** en el enlace por el identificador de dicho objeto.

Por su parte, la ruta asociada a este enlace podría ser algo así (en el archivo de rutas):

```
Route::get('/libros/{id}', [LibroController::class, 'show'])
->name('libros.show');
```

```
Aunque también podemos haber definido las rutas como un paquete de
    recursos, y cada una tendrá su método asociado:

Route::resource('libros', LibroController::class);
```

Finalmente, el método `show` del controlador asociado se encargará de obtener los datos del libro a partir de su `id`, y generar la vista correspondiente.

Para obtener los datos de un objeto a partir de su identificador, podemos emplear el método `find` del modelo, pasándole como parámetro el identificador. Así, podríamos generar una vista con los datos como ésta:

```
...
class LibroController extends Controller
{
    ...

    public function show($id)
    {
        $libro = Libro::find($id);
        return view('libros.show', compact('libro'));
    }
}
```



NOTA: si devolvemos (return) directamente lo que obtiene el método find, nos llegará al navegador en formato JSON. De hecho, si devolvemos un array, Laravel lo envía directamente en formato JSON. Esta característica la utilizaremos más adelante para definir servicios REST.

En el caso de que el objeto no se encuentre (porque, por ejemplo, utilicemos un id equivocado), la vista generada fallará. Para evitarlo, en lugar del método `find` podemos emplear `findOrFail`, que, en caso de que no se encuentre el objeto, generará una vista con un error 404, más apropiada. **Además, recuerda que puedes personalizar estas páginas de error definiendo las vistas correspondientes.**

```
$libro = Libro::findOrFail($id);
```

3.5 Inserciones

Las inserciones a través de Eloquent se pueden realizar creando una instancia del objeto, rellenando sus atributos y llamando al método `save`, heredado de la superclase Model.

```
$libro = new Libro();
$libro->titulo = "El juego de Ender";
$libro->editorial = "Ediciones B";
$libro->precio = 8.95;
$libro->save();
```

Como alternativa, también se puede utilizar el método `create` del modelo, y pasarle todos los datos de la petición, que llegarían desde un formulario, como veremos más adelante:

```
Libro::create($request->all());
```

Para que esto último funcione, deben cumplirse dos premisas:

- Cada campo de la petición debe tener asociado un campo del mismo nombre en el modelo.
- Debemos definir en el modelo una propiedad llamada `$fillable` con los nombres de los campos de la petición que nos interesa procesar (el resto se descartan). Esto es obligatorio especificarlo, aunque nos interesen todos los campos, para evitar inserciones masivas malintencionadas (por ejemplo, editando el código fuente para añadir otros campos y modificar datos inesperados).

```
class Libro extends Model
{
    protected $fillable = ['titulo', 'editorial', 'precio'];
}
```

Este código de inserción (o bien campo a campo, o usando el método `all`) se suele poner en el método `store` del controlador, para que reciba los datos del formulario de inserción y la haga en la base de datos.

3.6 Modificaciones

La modificación consiste en dos pasos:

- Encontrar el objeto a modificar (buscándolo por el id con findOrFail, por ejemplo, como se ha explicado antes)
- Modificar las propiedades que se necesiten, y llamar al método save del objeto para guardar los cambios.

Por ejemplo:

```
$libroAModificar = Libro::findOrFail($id);
$libroAModificar->titulo="Otro título";
$libroAModificar->save();
```

También podemos utilizar el método `update` enlazado con `findOrFail`, y pasarle como parámetro todos los datos de la petición, igual que se ha explicado para la inserción, y siempre y cuando hayamos declarado el atributo `$fillable` en el modelo para indicar qué campos se aceptan:

```
Libro::findOrFail($id)->update($request->all());
```

Este código de modificación se suele poner en el método `update` del controlador, para que reciba los datos del formulario de edición y haga la modificación correspondiente.



Figure 1: php artisan route:list

Como vemos en la imagen la llamada a *libros.update* y *libros.show* es igual, salvo por el protocolo, por lo tanto debereemos hacer un ajuste como el siguiente:

```
<form  action="{{ route('libros.update',$libro,$libro->id ) }}" method="
    POST">
   @csrf
   @method('PUT')
   ...
```

3.7 Borrados

Para hacer el borrado, también buscamos el objeto a borrar con `findOrFail`, y luego llamamos a su método `delete`:

```
Libro::findOrFail($id)->delete();
```

Esto lo haremos normalmente en el método `destroy` del controlador en cuestión. Después, podemos redirigir o renderizar alguna vista resultado, como el listado de libros general para comprobar que se ha borrado.

```
public function destroy($id)
{
    Libro::findOrFail($id)->delete();
    $libros = Libro::get();
    return view('libros.index', compact('libros'));
}
```

3.7.1 Sobre el borrado desde las vistas

Lo normal es que el borrado se active haciendo clic en algún elemento de una vista. Por ejemplo, haciendo clic en un botón o enlace que ponga "Borrar". Sin embargo, si implementamos esto así:

```
<a href="{{ route('libros.destroy', $libro }}">
Borrar
</a>
```

Si queremos borrar el libro con id 3, se generará una ruta `http://biblioteca/libros/3`. Lo podemos comprobar pasando el ratón por el enlace y viendo la barra inferior de estado del navegador. Esta ruta, sin embargo, nos va a enviar a la ficha del libro 3, no al borrado, ya que estamos enviando una petición GET, y no una de borrado (DELETE). Para evitar esto, la opción de borrado debe hacerse siempre desde un formulario, donde a través del helper `@method` indicamos que es una petición de borrado (DELETE).

Con lo que el "enlace" para borrar un libro quedaría así:

```
<form action="{{ route('libros.destroy', $libro) }}" method="POST">
    @method('DELETE')
    @csrf
    <button>Borrar</button>
</form>
```



NOTA el helper @csrf lo veremos con más detalle al hablar de formularios, pero se añade a los formularios Laravel para evitar ataques de tipo cross-site, es decir, accesos a una URL de nuestra web desde otras webs.